

```
graph LR; subgraph Row1; direction LR; A[Visual input  
t_k] --> B[Visual estimator]; B -- "Processing delay" --> C[Delayed correction  
t_k]; end; subgraph Row2; direction LR; D[IMU stream  
fast sampling] --> E[Inertial propagation]; end; E -- "Live estimate" --> C;
```

The diagram illustrates the system architecture. It consists of two main rows of components. The top row starts with a 'Visual input' block (containing a camera icon and  $t_k$ ), which feeds into a 'Visual estimator' block. The output of the 'Visual estimator' is a 'Delayed correction' block (containing  $t_k$ ), with the transition labeled 'Processing delay'. The bottom row starts with an 'IMU stream' block (containing an IMU icon and 'fast sampling'), which feeds into an 'Inertial propagation' block. The output of the 'Inertial propagation' block is a 'Live estimate', which is fed into the 'Delayed correction' block. The 'Delayed correction' block is the final output of the system.

The diagram illustrates the proposed state correction framework. It shows a sequence of states  $s(t_k)$ ,  $s(t_{k+1})$ , ...,  $s(t_{\text{now}})$  stored in a buffer. These states are updated using retained IMU samples  $(t_k, t_{\text{now}}]$  to produce corrected states  $s^+(t_k)$  and  $s^+(t_{\text{now}})$ . The process involves an 'Update' step, a 'Replay' step, and a 'Continue' step leading to the 'Next state'.

 $s^+(t)$  : corrected

Illustrative sensors and samples; symbolic timing, not to scale.

## Current pose